

Instructions

Objective

In this lab you will write a RISC-V assembly language program that reads a 32-bit **signed** integer input value from the console, interprets it as an unsigned 32-bit value with 14 fractional bits, computes its square root, and prints the output value to the console as a signed 32-bit value with 14 fractional bits.

Value Conversions

The user must manually convert the desired input value, called *input_actual*, to a (32,14) fixed-point representation, called *input_raw*, to enter into the program.

RARS only offers the ability to read a **signed** 32-bit integer, so any *input_raw* value greater than or equal to 2^{31} (2,147,483,648) but less than 2^{32} (4,294,967,296) will cause an error.

From the perspective of *input_actual*, this means that any actual value of $2^{32/2}14 = 2^{18}$ will require that the *input_raw* be entered as the negative complement.

These conversions are shown below.

(1) if *input_actual* < 131072 (2^{17}) then:

```
1 input_raw = floor(input_actual * 2^14)
```

(2) if *input_actual* >= 131072 (2^{17}) (and less than 262144 (2^{18})), then:

```
1 input_raw = floor(-(2^32 - (input_actual * 2^14)))
```

For example:

- if *input_actual* = 3.75 then *input_raw* = floor(3.75 * 16384) = 61440
- if *input_actual* = 150000, then *input_raw* = -1837367296

Internally your code must treat the input value as an unsigned fixed-point value with 14 fractional bits and 18 non-fractional bits.

Thus, if the value printed is named *output_raw*, then the actual output value, named *output_actual* is computed as:

(1) $2147483648 < \text{output_raw} \leq 0$, then:

```
1 output_actual = output_raw / 2^14
```

(2) $-2147483648 < \text{output_raw} \leq -1$, then:

```
1 output_actual = (2^32 + output_raw) / 2^14
```

Note that your code should always produce an actual output value that is less than $\sqrt{2^{18}} = 512$, which corresponds to an raw output value of 8388608 (i.e. the raw output should never be negative).

Your code should produce the following outputs for the example inputs shown in the table below.

Input raw	Input actual	Output actual	Output raw
16384	1.0	1.0	16384
0	0.0	0.0	0.0
-16384	262143	~511.99902	8388592
-1	~262143.99999	~511.99988	8388607
409600	25	5	81920
112558	~6.869995	2.62097	42943

Your code should run in the RARS simulator.

Additional Considerations

- Please do not print prompts or other messages from your program; only the raw result value.
- You are not required to check if the input value is negative, since negative inputs actually correspond to large positive inputs.
- We recommend you use an initial guess of 0 and an initial step of 256 (as an actual value). Also, note that comparing the x^2 value (square of the guess) as a (32,14) value as opposed to a (64,28) value will cause a loss in accuracy, but you will not be penalized for this.
- You are not required to use a binary search algorithm. However, since our grading rubric is based on binary search, you will not receive partial credit for rubric points (B) to (F) if you use another algorithm, such as Newton-Raphson or CORDIC.

Deliverables

- Submit your RISC-V program as a single ASCII text file with name "sqrt.asm".
- Each group only needs to submit once via either partner.

Rubric

- (A) 10 points – reads input value.
- (B) 10 points – squares and shifts guess.
- (C) 20 points – compares guess with input value with outcomes to $<$, $>$, and $=$.
- (D) 20 points – adds or subtracts guess.
- (E) 10 points – shifts step.
- (F) 20 points – loops until step becomes 0.
- (G) 10 points – prints final guess.

Maximum score: 100 points.